STAT:5400 Tech Guide 3 — R Packages

Boxiang Wang

2026-10-28

R relies heavily on its packages. A package contains code, data, documentation, and vignettes (long documents that show how to use the package). As of October 2026, there are more than 25,000 packages on CRAN, the Comprehensive R Archive Network (https://cran.r-project.org/).

In this class, you have installed many packages with `install.packages`. It is time for us to build our own R package. It is a good habit to organize our code in a package. The code becomes more standardized, which saves our time, and it is easier to share.

- A package is distributed as a `.tar.gz` file, a compressed archive of all its files.
- We use the function `pvalDist2` from Tech Guide 1 as the example.
- Writing R packages is not on the midterm.

1 Installing packages

1.1 From CRAN

`install.packages` downloads a package from CRAN and installs it. `library` loads it. We install a package once, and we load it in every R session.

```
install.packages("profvis")
library(profvis)
packageVersion("profvis")
```

R keeps the installed packages in one or more **library** folders. To see them:

```
.libPaths()
```

New packages go to the first folder. To remove a package, use `remove.packages("profvis")`. To update all packages, use `update.packages()`.

Note that on Windows and Mac, CRAN provides compiled (binary) packages, so the installation is fast. On Linux, R compiles the C, C++, or Fortran code of every package on your machine. Installing a large package can take a few minutes.

1.2 On the lnx machines

On the lnx machines, we cannot write to the system library. The first time we run `install.packages`, R asks

```
Would you like to use a personal library instead? (yes/No/cancel)
Would you like to create a personal library ... to install packages into? (yes/No/cancel)
```

Answer `yes` to both. R creates a folder under `~/R` and installs the packages there. In R in a terminal, R may also ask us to choose a CRAN mirror. Type `1` and press Enter.

We can also create the personal library ourselves. In R:

```
Sys.getenv("R_LIBS_USER")                    # the personal library
dir.create(Sys.getenv("R_LIBS_USER"), recursive = TRUE, showWarnings = FALSE)
```

Then restart R. Now `.libPaths()` lists the personal library first, and `install.packages` uses it.

1.3 From GitHub

Many packages are on GitHub before they go to CRAN. The package `remotes` installs them. We use the form `remotes::install_github`, which calls the function without loading the package.

```
install.packages("remotes")
remotes::install_github("rladies/praise")
praise::praise()
```

1.4 From a file

A package can also come as a `.tar.gz` file. We install it with `install.packages("file.tar.gz", repos = NULL, type = "source")` in R, or with `R CMD INSTALL file.tar.gz` in a terminal. We will do this below with our own package.

1.5 Build tools

To build our own package, and to install a package with C, C++, or Fortran code from source, we need compilers.

- Windows: install Rtools. Choose the version that matches your R, for example Rtools 4.5 for R 4.5 and later, and Rtools 4.4 for R 4.4. No other setup is needed.
- Mac: open Terminal and run `xcode-select --install`. This installs the Xcode command line tools, which include the C and C++ compilers. After a macOS upgrade, run it again: old tools give errors such as `invalid value 'gnu++17'`. For Fortran, install the gfortran package from https://mac.r-project.org/tools/.
- Linux, including the lnx machines: the compilers are usually installed already.

We check with the package `devtools`. The first line installs `devtools` only if it is missing.

```
if (!requireNamespace("devtools", quietly = TRUE)) install.packages("devtools")
devtools::has_devel()
```

It prints `Your system is ready to build packages!` if the compilers are found. If you have trouble with the compilers, log in to an lnx machine and build your package there.

2 Writing an R package

2.1 The package `devtools`

We use the package `devtools`. It loads the package `usethis` as well. The two packages do the routine work for us: `create_package` makes the folders, `document` writes the documentation, and `build`, `install`, and `check` build, install, and check the package.

```
library(devtools)
```

2.2 The structure of an R package

An R package has the following components.

- `DESCRIPTION`: the metadata of the package, such as the title, version, authors, description, license, and the packages it depends on.
- `NAMESPACE`: the functions the package exports, and the functions it imports from other packages.
- `R/`: the R code.
- `man/`: the documentation, one `.Rd` file per function.

- `src/`: the C, C++, or Fortran code, if any.

A package may also have other folders, such as `data/`, `vignettes/`, and `tests/`.

2.3 Creating an R package

We use the function `pvalDist2` from Tech Guide 1. It simulates `B` samples of size `n` from a distribution and returns the `B` p-values of the one-sample t test. We drop the unused argument `alp`. Our goal is a package `pvalDist` with this function.

We simply name the package after the function. When you create a real package, try your best to find a good name, which is easy to remember and self-explanatory.

We first set a working directory. Change `mydir` to your own folder. The package will be created inside it.

```
mydir <- "~/STAT5400/package"
dir.create(mydir, recursive = TRUE, showWarnings = FALSE)
setwd(mydir)
```

1. We create a folder `pvalDist` with the minimal files of an R package using `create_package`. If the folder exists, we delete it first.

```
if (dir.exists("pvalDist")) unlink("pvalDist", recursive = TRUE)
create_package("pvalDist", open = FALSE)
list.files("pvalDist", all.files = TRUE, no.. = TRUE)
```

We now have a folder `R` and the files `DESCRIPTION` and `NAMESPACE`. Below is the `DESCRIPTION` file that we edit based on the automatically generated file. We write it from R with `cat`. The form `r"---( ... )---"` is a raw string: everything between the brackets is kept as it is, including quotes and backslashes.

```
cat(file = "pvalDist/DESCRIPTION", r"---(Package: pvalDist
Type: Package
Title: Simulated P-Values of the One-Sample t Test
Version: 0.0.1
Date: 2026-10-28
Authors@R: c(person(given = "Boxiang", family = "Wang",
                    role = c("aut", "cre"), email = "boxiang-wang@uiowa.edu"),
             person(given = "Indiana", family = "Jones", role = "aut"))
Description: Simulates the p-values of the one-sample t test when the data
    come from a user-specified distribution. The package is used in
```

```
    STAT:5400 to show how to create an R package.
License: GPL-3
Encoding: UTF-8
Imports: stats
Roxygen: list(markdown = TRUE)
)---")
```

For the version `0.0.1`:

- The first number is the major version. It changes when the package changes in a way that breaks old code, for example `0.0.1` to `1.0.0`.
- The second number is the minor version. It changes when we add features and the old code still works, for example `0.1.0` to `0.2.0`.
- The third number is the patch. It changes for small fixes, for example `0.0.1` to `0.0.2`.

2. We put the R file into the folder `pvalDist/R`. The comments that start with `#'` are **roxygen** comments. They have several parts, such as `@param`, `@return`, `@examples`, and `@export`. The package `roxygen2` turns them into the documentation and the `NAMESPACE` file, so we never edit those by hand.

```
cat(file = "pvalDist/R/pvalDist2.R", r"---(#' Simulated p-values of the one-sample t test
#'
#' Simulates `B` samples of size `n` from a distribution and returns the
#' `B` p-values of the two-sided one-sample t test of `mu_0`.
#'
#' @param n Sample size.
#' @param dist Name of the distribution without the leading `r`, for
#'   example `"norm"` or `"exp"`.
#' @param mu_0 The population mean under the null hypothesis.
#' @param B Number of simulated samples.
#' @param ... Further arguments passed to the random number generator,
#'   for example `rate = 1/2`.
#' @return A vector of `B` p-values.
#' @examples
#' set.seed(5400)
#' p <- pvalDist2(n = 10, dist = "exp", mu_0 = 2, B = 1000, rate = 1/2)
#' mean(p < 0.05)   # Type I error
#' @importFrom stats pt sd
#' @export
pvalDist2 <- function(n, dist, mu_0, B = 10000, ...) {
  # simulate data from user-specified dist
  Xlist <- do.call(paste0("r", dist), list(B * n, ...))
```

```
  Xlist <- matrix(Xlist, B, n)
  # compute t-statistic for each sample
  Xbarlist <- rowMeans(Xlist)
  sdlist <- apply(Xlist, 1, sd)
  t_stat <- (Xbarlist - mu_0) / (sdlist / sqrt(n))
  # returns the observed p-values
  pt(-abs(t_stat), n - 1) + 1 - pt(abs(t_stat), n - 1)
}
)---")
```

Note that the function uses `pt` and `sd` from the package `stats`. The line `@importFrom stats pt sd` declares this in `NAMESPACE`, and `Imports: stats` declares it in `DESCRIPTION`. `R CMD check` reports a problem if we forget either one.

3. We now generate the documentation and the `NAMESPACE` file.

```
document("pvalDist")
cat(readLines("pvalDist/NAMESPACE"), sep = "\n")
```

A folder `man` is generated with the file `pvalDist2.Rd` inside it. The `NAMESPACE` file now exports `pvalDist2` and imports `pt` and `sd`.

4. (Optional) We add a **vignette**, a long-form document that shows how to use the package. Building a vignette needs pandoc, a program that converts documents between formats. RStudio and Positron come with it. Run `rmarkdown::pandoc_available()` first. If it is `FALSE`, skip this step.

   `use_vignette` creates a template `vignettes/pvalDist.Rmd` and adds `knitr` and `rmarkdown` to `DESCRIPTION`. We keep the template as it is. `R CMD build` in the next step builds it. Note that in R in a terminal, for example on the lnx machines, `use_vignette` opens the new file in an editor. If it opens nano, press `Ctrl-X` to return to R.

```
rmarkdown::pandoc_available()
proj_set("pvalDist")
use_vignette("pvalDist")
```

5. We now build, install, and check the package. These are shell commands. We run them from R with `system`, in the folder `mydir` that contains `pvalDist`. You may also type them in a terminal.

```
system("R CMD build pvalDist")
system("R CMD INSTALL pvalDist_0.0.1.tar.gz")
```

`R CMD build` makes the file `pvalDist_0.0.1.tar.gz`, where `0.0.1` is the version in `DESCRIPTION`. `R CMD INSTALL` installs it into our library. From R, `build("pvalDist")` and `install("pvalDist")` do the same.

`R CMD check` tests the package: the code, the documentation, the examples, and the vignette. Fix all errors and warnings. Add `--as-cran` if you are going to submit the package to CRAN.

```
system("R CMD check pvalDist_0.0.1.tar.gz")
```

We test our package:

```
library(pvalDist)
packageVersion("pvalDist")
set.seed(5400)
p <- pvalDist2(n = 10, dist = "exp", mu_0 = 2, B = 1000, rate = 1/2)
mean(p < 0.05)
```

Try `?pvalDist2` to see the help page. Submit your package to CRAN, https://cran.r-project.org/submit.html, when you feel it is ready.

3 Extending R with C++ and Fortran

For large-scale computation, we can write the main algorithm of a package in C++ or Fortran. The code runs much faster and uses less memory.

`pvalDist2` computes the t statistic of each sample. We now write a function for the t statistic of one sample in R, C++, and Fortran, and compare the speed. The R version:

```
tstat_R <- function(x, mu_0) {
  n <- length(x)
  (mean(x) - mu_0) / (sd(x) / sqrt(n))
}
```

3.1 C++

The R package `Rcpp` connects R and C++. We write the C++ version into the file `tstat_C.cpp` in `mydir`. The line `// [[Rcpp::export]]` tells `Rcpp` to make the function available in R. The comments that start with `//'` are roxygen comments. They are used when the file goes into a package. The line `@useDynLib pvalDist, .registration = TRUE` tells R to load the compiled code of the package `pvalDist`. The line `@importFrom Rcpp sourceCpp` makes sure that `Rcpp` is loaded too.

```
cat(file = "tstat_C.cpp", r"---(#include <Rcpp.h>
using namespace Rcpp;

//' One-sample t statistic (C++)
//'
//' @param x A numeric vector.
//' @param mu_0 The population mean under the null hypothesis.
//' @return The t statistic.
//' @examples
//' tstat_C(c(1, 2, 3, 5), mu_0 = 2)
//' @useDynLib pvalDist, .registration = TRUE
//' @importFrom Rcpp sourceCpp
//' @export
// [[Rcpp::export]]
double tstat_C(NumericVector x, double mu_0) {
  int n = x.size();
  double s = 0.0, ss = 0.0;
  for (int i = 0; i < n; i++) s += x[i];
  double xbar = s / n;
  for (int i = 0; i < n; i++) ss += (x[i] - xbar) * (x[i] - xbar);
  double sd = sqrt(ss / (n - 1));
  return (xbar - mu_0) / (sd / sqrt(n));
}
)---")
```

`Rcpp::sourceCpp` compiles the file and creates the R function `tstat_C`.

```
if (!requireNamespace("Rcpp", quietly = TRUE)) install.packages("Rcpp")
Rcpp::sourceCpp("tstat_C.cpp")
x <- c(1, 2, 3, 5)
tstat_R(x, mu_0 = 2)
tstat_C(x, mu_0 = 2)
```

3.2 Fortran

We write the Fortran version into the file `tstat_f.f90`. A Fortran subroutine has no return value. The result goes into the last argument `t`.

```
cat(file = "tstat_f.f90", r"---(subroutine tstat_f(n, x, mu0, t)
  implicit none
  integer, intent(in) :: n
```

```
  double precision, intent(in) :: x(n), mu0
  double precision, intent(out) :: t
  integer :: i
  double precision :: xbar, ss
  xbar = 0.0d0
  do i = 1, n
    xbar = xbar + x(i)
  end do
  xbar = xbar / dble(n)
  ss = 0.0d0
  do i = 1, n
    ss = ss + (x(i) - xbar)**2
  end do
  t = (xbar - mu0) / (sqrt(ss / dble(n - 1)) / sqrt(dble(n)))
end subroutine tstat_f
)---")
```

`R CMD SHLIB` compiles the file into a shared library, `tstat_f.so` on Mac and Linux and `tstat_f.dll` on Windows. `dyn.load` loads it into R, and `.Fortran` calls the subroutine. We write an R wrapper, so that the function looks like an ordinary R function.

```
system("R CMD SHLIB tstat_f.f90")
dyn.load(paste0("tstat_f", .Platform$dynlib.ext))
tstat_F <- function(x, mu_0) {
  .Fortran("tstat_f", n = length(x), x = as.double(x), mu0 = as.double(mu_0),
    t = 0, PACKAGE = "tstat_f")$t
}
tstat_F(x, mu_0 = 2)
```

3.3 Comparing the speed

We compare the three versions using the package `microbenchmark`. The option `check = "equal"` makes sure that the three results agree.

```
if (!requireNamespace("microbenchmark", quietly = TRUE)) install.packages("microbenchmark")
library(microbenchmark)
set.seed(5400)
x <- rnorm(10)
microbenchmark(tstat_R(x, 0), tstat_C(x, 0), tstat_F(x, 0), check = "equal")
```

Increase the sample size:

```
x <- rnorm(1000)
microbenchmark(tstat_R(x, 0), tstat_C(x, 0), tstat_F(x, 0), check = "equal")
```

3.4 R packages with C++

We now add `tstat_C` to the package. We first change the version in `DESCRIPTION` to `0.0.2`.

```
d <- read.dcf("pvalDist/DESCRIPTION")
d[, "Version"] <- "0.0.2"
write.dcf(d, "pvalDist/DESCRIPTION")
```

We now prepare the package for `Rcpp`. `use_package` adds `Rcpp` to `LinkingTo` and `Imports` in `DESCRIPTION`. We also create the folder `src` for the C++ code.

```
proj_set("pvalDist")
use_package("Rcpp", "LinkingTo")
use_package("Rcpp")
dir.create("pvalDist/src")
```

Note that `usethis` also has the function `use_rcpp`. Without a file name, it often stops with an error, so we do not use it here.

We copy the C++ file into `src` and run `compileAttributes`. It writes the R wrapper `R/RcppExports.R` and the file `src/RcppExports.cpp` for us. The roxygen comments in the C++ file go into the R wrapper, so `document` uses them and adds `useDynLib(pvalDist, .registration = TRUE)` to `NAMESPACE`.

```
file.copy("tstat_C.cpp", "pvalDist/src/tstat_C.cpp")
Rcpp::compileAttributes("pvalDist")
document("pvalDist")
```

We build, install, and check the package again.

```
system("R CMD build pvalDist")
system("R CMD INSTALL pvalDist_0.0.2.tar.gz")
system("R CMD check pvalDist_0.0.2.tar.gz")
```

We test the package. Restart R first, because the old version `0.0.1` is still loaded. In RStudio, use Session > Restart R. In a terminal, type `q("no")` and start R again. A restart may keep old objects, such as `tstat_C` from `sourceCpp`. They would hide the functions of the package, so we

remove them with `rm(list = ls())`. Then we load `devtools` and set the working directory again.

```
rm(list = ls())
library(devtools)
mydir <- "~/STAT5400/package"
setwd(mydir)
library(pvalDist)
packageVersion("pvalDist")
tstat_C(c(1, 2, 3, 5), mu_0 = 2)
```

Note that `Rcpp.package.skeleton("mypkg")` creates a whole package with `Rcpp` directly. Follow the file `Read-and-delete-me` inside it.

3.5 R packages with Fortran

We change the version to `0.0.3` and copy the Fortran file into `src`.

```
d <- read.dcf("pvalDist/DESCRIPTION")
d[, "Version"] <- "0.0.3"
write.dcf(d, "pvalDist/DESCRIPTION")
file.copy("tstat_f.f90", "pvalDist/src/tstat_f.f90")
```

We write the R wrapper into `R/tstat_F.R`, with roxygen comments. The `PACKAGE` argument is now the name of our package.

```
cat(file = "pvalDist/R/tstat_F.R", r"---(#' One-sample t statistic (Fortran)
#'
#' @param x A numeric vector.
#' @param mu_0 The population mean under the null hypothesis.
#' @return The t statistic.
#' @examples
#' tstat_F(c(1, 2, 3, 5), mu_0 = 2)
#' @export
tstat_F <- function(x, mu_0) {
  .Fortran("tstat_f", n = length(x), x = as.double(x), mu0 = as.double(mu_0),
    t = 0, PACKAGE = "pvalDist")$t
}
)---")
```

R needs to know the names of the compiled routines. This is called **registration**. `Rcpp` registered `tstat_C` for us. With Fortran, we write the file `src/init.c` ourselves. It lists the C++ routine and the Fortran routine. Change the names and the numbers of arguments for your own package.

```
cat(file = "pvalDist/src/init.c", r"---(#include <R.h>
#include <Rinternals.h>
#include <R_ext/Rdynload.h>

/* the C++ routine made by Rcpp */
extern SEXP _pvalDist_tstat_C(SEXP xSEXP, SEXP mu_0SEXP);

/* the Fortran subroutine */
extern void F77_NAME(tstat_f)(int *n, double *x, double *mu0, double *t);

static const R_CallMethodDef CallEntries[] = {
  {"_pvalDist_tstat_C", (DL_FUNC) &_pvalDist_tstat_C, 2},
  {NULL, NULL, 0}
};

static const R_FortranMethodDef FortranEntries[] = {
  {"tstat_f", (DL_FUNC) &F77_NAME(tstat_f), 4},
  {NULL, NULL, 0}
};

void R_init_pvalDist(DllInfo *dll) {
  R_registerRoutines(dll, NULL, CallEntries, FortranEntries, NULL);
  R_useDynamicSymbols(dll, FALSE);
}
)---")
```

We run `compileAttributes` again. It sees our `init.c` and no longer writes its own registration. Then we update the documentation, and build, install, and check the package.

```
Rcpp::compileAttributes("pvalDist")
document("pvalDist")
system("R CMD build pvalDist")
system("R CMD INSTALL pvalDist_0.0.3.tar.gz")
system("R CMD check pvalDist_0.0.3.tar.gz")
```

We test the three functions. Restart R first, because the old version `0.0.2` is still loaded. As before, we remove the old objects first.

```r
rm(list = ls())
library(pvalDist)
packageVersion("pvalDist")
x <- c(1, 2, 3, 5)
tstat_C(x, mu_0 = 2)
tstat_F(x, mu_0 = 2)
set.seed(5400)
p <- pvalDist2(n = 10, dist = "exp", mu_0 = 2, B = 1000, rate = 1/2)
mean(p < 0.05)
```

Read *Writing R Extensions*, https://cran.r-project.org/doc/manuals/r-devel/R-exts.html, for more information.

4 Exercises

✋ By hand

1. Add a function `TypeIErr(n, dist, mu_0, alp = 0.05, B = 10000, ...)` to the package. It returns `mean(pvalDist2(n, dist, mu_0, B, ...) < alp)`. Write the roxygen comments, run `document`, and rebuild and check the package.
2. Rewrite `pvalDist2` so that it computes the t statistic of each row with `tstat_C`. Compare the speed with the original version using `microbenchmark`. Is it faster? Why or why not?
3. Copy `pvalDist_0.0.3.tar.gz` to an lnx machine with `scp`. Install it there with `R CMD INSTALL` into your personal library, and run the example of `pvalDist2`.

5 References

- Hadley Wickham and Jennifer Bryan, *R Packages*, 2nd edition.
- R Core Team, *Writing R Extensions*.
- Dirk Eddelbuettel and others, *Rcpp: Seamless R and C++ Integration*. In R, type `vignette("Rcpp-introduction")`.